

选题	2024 年第十四届 APMCM	参赛编号
C 题	亚太地区大学生数学建模竞赛（中文赛项）	apmcm 24102422

基于量子计算的物流配送问题

摘要

随着电子商务的蓬勃发展，物流配送的复杂性和高效性要求日益提升。本文基于物流公司需应对多公司、多城市、多货物类型的复杂运输场景，基于 QUBO 模型研究物流公司物流配送最小化成本制定最优运输模型。首先，针对不同问题分别构建以最小化成本为目标的组合优化模型。其次，根据不同目标函数形式，通过采用启发式方法等数学技巧进行转换，转换成 QUBO 形式，并运用量子退火算法并通过 Kaiwu SDK 进行问题求解出使得最终成本最小化时的组合货车租赁方案。最后，本文运用罚函数对量子退火算法的结果进行对比检验分析，凸显量子退火算法优越性并总结不同算法优劣及适用场景我们综合了前三段的研究成果，强调了量子计算在物流优化与人工智能模型超参数调优中的重要作用。同时，将量子计算应用于人工智能模型的超参数调优，进一步展示了其在提升模型性能、加速科研进程方面的巨大潜力。

针对问题一：两个物流公司独立运营且拼货只在公司内部，以最小化单个物流公司运营成本为目标建立 QUBO 模型，使用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器分别求解。计算得出租赁 1 辆卡车 a 和 2 辆卡车 b 时总成本最小，经罚函数灵敏度分析确定罚参数并通过贪心算法检验结果合理性，最终确定租赁 1 辆卡车 a 和 2 辆卡车 b 为货车租赁的最优方案及相应货物运输方案。

针对问题二：在问题一基础上修改参数，考虑运输成本等因素建立目标函数和约束条件，再进行物流转运中心选址以达到最短路径降低成本的目的。运用数学规划方法和改进麻雀优化算法，使用 subQUBO 模型和量子退火算法，通过 KaiwuSDK 求解货物运输方案和成本，迭代确定最优物流中心坐标。随机抽样比较总成本以验证量子退火算法的优越性，最终通过 subQUBO 方法和模拟退火算法得出卡车租赁和货物运输的最优方案及相关成本信息。

针对问题三：选取“人工智能模型超参数调优”场景，以超参数为决策变量、模型性能度量为目标函数构建 QUBO 模型，引入二进制变量并设计目标函数及引入惩罚因子，期望通过量子计算实现优化速度、超参数调整、高维空间问题解决、局部最优解克服等，通过数据探究得到 QUBO 求解算法在不同参数配置下的准确度分布情况，准确率总体较高且有变动。计算得出，引入量子计算至 AI 超参数调优可以加速搜索过程，缩短调优时间，并精准定位最优超参数以提升模型性能与泛化能力。其特别优势在于有效处理高维空间优化难题，克服传统算法的局限，同时增强全局搜索能力，避免局部最优陷阱，增强模型稳定性。

关键词：物流配送、量子计算、QUBO 模型、KaiwuSDK、CIM 模拟器、模拟退火求解器、subQUBO 模型、物流转运中心选址、麻雀优化算法、人工智能模型超参数调优。

目录

一、问题重述	1
1.1 研究背景	1
1.2 问题重述	1
二、问题分析	1
2.1 问题一的分析	1
2.2 问题二的分析	2
2.3 问题三的分析	2
三、模型假设	2
四、符号说明	2
五、问题一模型的建立与求解	3
5.1 模型的建立	4
5.1.1 QUBO 模型	4
5.1.2 QUBO 模型的建立	5
5.1.3 罚函数灵敏度分析	6
5.2 模型的求解	7
5.2.1 量子计算	7
5.2.2 算法设计与求解	8
5.2.3 问题一的求解与最优方案	8
5.3 最优方案	8
5.3.1 货车租赁方案	8
5.3.2 货物运输方案	8
六、问题二模型的建立与求解	9
6.1 模型的建立	9
6.1.1 QUBO 模型的建立	9
6.1.2 罚函数灵敏度分析	9
6.1.3 物流转运中心选址	10
6.1.4 改进麻雀优化算法	11
6.2 模型的求解	13
6.2.1 subQUBO 模型的求解	13
6.2.2 罚函数灵敏度分析	13
6.2.3 物流转运中心选取	14
6.2.4 最优化成本	15
6.3 复杂组合成本最优化方案	15
6.3.1 货车租赁方案	15
6.3.2 货物运输方案	16
七、问题三模型的建立与求解	16
7.1 人工智能模型超参数调优的 QUBO 模型	16
7.1.1 人工智能超参调优应用	16
7.1.2 背景信息	17
7.1.3 研究方法	17
7.1.4 思路及技术路线	17
7.1.5 QUBO 模型设计	17
7.2 研究结果	18
7.2.1 QUBO 模型算法	18
7.2.2 超参数调优 QUBO 模型	18
七、模型的评价与推广	19
7.1 模型的优点	19
7.2 模型的缺点	19
7.3 模型的推广	20
参考文献	21
附录	22

一、问题重述

1.1 研究背景

随着电商的蓬勃发展，物流配送需求激增，物流公司与电商平台紧密合作以应对挑战。面对庞大的货物量与多样化的目的地，传统物流优化方法显得力不从心。为此，物流公司探索采用量子计算技术这一前沿解决方案，旨在自动化计算运输综合策略^[1]。量子计算凭借其强大的计算能力，有望为物流公司提供更精准、高效的路线规划与运输方式选择，确保货物准时送达，同时优化成本结构。此举不仅能显著提升物流效率，降低运营成本，还能增强消费者的购物体验与满意度，为电商行业的持续发展注入新动力。

借助量子计算技术来解决物流配送中的优化问题具有极其重要的现实意义和深远的应用价值^[2]。它能够为物流公司提供更为科学、高效的决策支持，有力地推动物流行业朝着更加智能化、高效化的方向发展。这不仅有助于提升物流公司的竞争力，还能对整个电商行业的繁荣发展提供坚实的保障。

1.2 问题重述

问题一：假设两个物流公司独立运营，且拼货只发生在公司内部。以最小化单个物流公司的运营成本为目标，建立 QUBO 模型。使用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器分别进行求解，为两个物流公司分别设计出货车租赁方案和货物运输方案。

问题二：当两个物流公司之间进行合作运营时，公司之间可以拼货运输，此时的优化目标为最小化两个公司的总成本。运用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器来求解，给出最优的货车租赁方案和货物运输方案，并计算出合作运营带来的总体成本减少量。

问题三：自行提出一个具有商业化前景或学术价值的场景，该场景可以涉及 AI、通信、金融、生物医学、物流供应链管理等相关领域。需要给出相应的 QUBO 模型表达式，并计算模型所需的比特数量级（可以用相关参数表示）。

二、问题分析

2.1 问题一的分析

在这个问题中，我们假设两个物流公司是独立运营的，并且拼货只在各自公司内部进行。我们的目标是最小化单个物流公司的运营成本。为了实现这个目标，我们需要建立一个 QUBO 模型。这个模型将考虑到各种因素，比如货物的起始位置、目的地、卡车的类型和载重、租赁时长、运输路线以及运输成本等。通过对这些因素的综合分析，我们可以确定最佳的货车租赁方案和货物运输方案。在建立模型的过程中，我们需要用合适的数学表达式来描述这些因素之间的关系，并且确保模型能够准确地反映实际情况。然后，我们使用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器来求解这个模型^[3]。这两个求解器将根据模型的设定，尝试找到最优的解决方案。通过这个过程，我们可以为每个物流公司设计出最合理的货车租赁和货物运输计划，从而在满足运输需求的同时，最大限度地降低运营成本。

2.2 问题二的分析

当两个物流公司决定合作运营时，情况就发生了变化。此时，公司之间可以进行拼货运输，我们的优化目标也变成了最小化两个公司的总成本。为了达到这个目标，我们同样需要建立一个 QUBO 模型。这个模型将考虑到两个公司的所有货物和运输需求，以及它们之间的协同合作方式。在模型中，我们需要考虑如何合理地安排货物的运输路径，如何充分利用拼货运输的优势，以及如何优化卡车的租赁和使用，以达到总成本最小化的目的。然后，我们使用 Kaiwu SDK 中的 CIM 模拟器和模拟退火求解器来求解这个模型^[4]。通过这两个求解器的计算，我们可以找到最优的货车租赁方案和货物运输方案，使得两个公司的总成本降到最低。同时，我们还可以计算出合作运营带来的总体成本减少量，这将有助于我们评估合作的效益和优势。

2.3 问题三的分析

在这个问题中，我们需要自行提出一个具有商业化前景或学术价值的场景。这个场景可以涉及到多个领域，比如 AI、通信、金融、生物医学、物流供应链管理等。然后，我们需要为这个场景建立一个相应的 QUBO 模型表达式。在建立模型表达式时，我们需要深入分析场景中的各种因素和关系，确定合适的变量和约束条件，并用数学语言来准确地描述它们。同时，我们还需要计算模型所需的比特数量级。这个比特数量级可以用相关的参数来表示，它将反映模型的复杂度和规模。通过提出这个场景和建立相应的模型，我们可以展示 QUBO 模型在不同领域的应用潜力和创新可能性，为解决实际问题提供新的思路和方法。

三、模型假设

- 1.假设每个城市有足够数量的可供租赁的卡车，且卡车的租赁和运输过程中不会出现故障或延误。
- 2.假设题目所给的货物起始城市、目的地城市、货物数量、卡车运输时间和成本等数据真实可靠，且在运输过程中保持不变。
- 3.假设仅考虑如何实现最小化运营成本或总成本的最优方案，不考虑其他因素对结果的影响。
- 4.假设物流公司在运输过程中，货物不会出现损坏或丢失等情况，且运输时间严格按照给定的时间表进行。
- 5.假设在计算成本时，只考虑卡车租赁费用、运输成本和航空运输费用，不考虑其他额外费用。

四、符号说明

变量	解释说明
A	表示 12 吨载重卡车的型号
B	表示 5 吨载重卡车的型号
P	表示飞机的型号
wki	表示第 k 种运输工具在数量为 i 时的二进制值
tki	表示第 k 种运输工具的数量，即：tki
cA	12 吨载重卡车的租赁价格，5000 元/天
cB	5 吨载重卡车的租赁价格，3000 元/天
cP	飞机的租赁价格，10000 元/吨
dA	12 吨载重卡车从上海到西安的单趟成本
dB	5 吨载重卡车从上海到西安的单趟成本

dP	货物从上海到西安的航空运价，假设为 10000 元/吨
Y	运输物流中投入使用的不同载量运输工具型号 a, b 数量
x	决策变量，用于表示是否租赁运输工具，0 表示不租赁，1 表示租赁
y	决策变量，用于表示货物是否通过航空运输到城市，0 表示不通过航空运输，1 表示通过航空运输
z	决策变量，用于表示其他相关决策，具体含义根据问题而定
q	货物的数量
s	运输工具的货箱容量
y	运输工具的效率
r	运输工具的油耗
p	运输工具的维护成本
i	表示迭代次数或其他相关计数
α	系统参数，在特定公式中使用
β	麻雀算法中的相关参数
Q	QUBO 模型中的系数矩阵
λ	正标量惩罚项，在 QUBO 模型中用于调整目标函数
Γ	量子退火算法中的场强，促使自旋状态发生转变
M	重心计算公式中的中间变量，用于表示麻雀种群的平均位置
Ne	能量水平
Ns	可解的个数
Ni	量子位的数量

五、问题一模型的建立与求解

综合上述问题分析，我们得到思路流程图如图 5.1 所示：

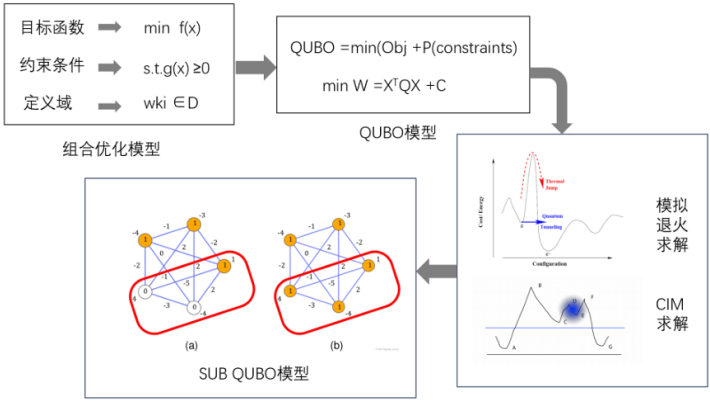


图 5.1 思路流程图

其中我们将 QUBO 模型看作一个灰盒，将题中表格的数据放入盒中针对具体问题求解。QUBO 模型求解具体流程图如图 5.1 所示：

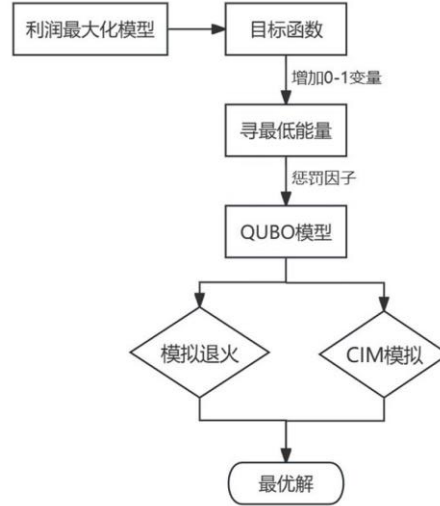


图 5.2QUBO 模型求解具体流程图

5.1 模型的建立

5.1.1QUBO 模型

QUBO 模型可以表述众多组合优化问题，本题中的最优租赁方案问题满足其要求。QUBO 无约束二次二进制优化模型作为量子计算中应用最为普遍的优化手段，它成功地整合了众多复杂的组合优化问题。当面临问题规模庞大、难以通过精确求解器找到“最优”解的情况时，该模型便能够派上用场。它采用启发式方法，在有限的时间约束下努力寻找尽可能接近最优的方案，为复杂问题的求解提供了一种高效且实用的途径。

QUBO 模型的最简易形式可表述为：

$$y_{min} = z^T P z \quad (5.1)$$

其中 z 作为二进制变量组成的向量，即当中每个变量的取值均为 $\{0,1\}$ ，在 QUBO 模型中，为了使得约束条件得到更好的表示，我们引入二进制 0-1 变量。QUBO 模型是解决组合优化问题的数学形式，而在模型中，变量通常被限制为二进制（0 或 1），这是因为 QUBO 模型通常用于描述决策问题，其中每个变量代表一个决策变量，其取值为 0 或 1，分别表示不做某项决策或做某项决策。

QUBO 目标为找到使得 y 最小的 z 。P 是常数的方阵，并且该常数方阵一般在最优化问题中，为对称矩阵或上/下三角形形式。

$$\begin{cases} q'_{ij} = (q_{ij} + q_{ji}), i < j \\ q'_{ij} = 0, i > j \end{cases} \quad (5.2)$$

表 5.1 租赁价格

型号	12 吨载重卡车	5 吨载重卡车	飞机
租赁价格	5000	3000	10000

根据此表，按对应货物数量计算卡车租赁方案的最小成本。

假设 w_{ki} 为第 k 种卡车在数量为 i 时的二进制值， t_{ki} 表示第 k 种卡车的数量，即： $t_{ki} = i$ 。因此，得到约束函数表达式：

$$\sum_{i=1}^{24} 100w_{1i}t_{1i} + 140w_{2i}t_{2i} + 200w_{3i}t_{3i} + 320w_{4i}t_{4i} = 2400 \quad (5.3)$$

将约束条件转化为:

$$\begin{aligned} \min f(z) \\ \text{s.t. } Az = b \\ f'(z) = f(z) + \lambda(Az - b)^T(Az - b) \end{aligned} \quad (5.4)$$

其中 λ 为正标量惩罚项,为一个足够大的数。

随问题规模的扩大,传统计算方法无法有效地计算出结果,此时使用适配于相干伊辛机(CoherentIsingMachine,CIM)的 QUBO 模型,能够在量子计算机硬件上进行毫秒级加速,从而实现最优解结果的快速得出。

5.1.2 QUBO 模型的建立

根据题意,我们知道约束条件包括对于每个物流公司租赁卡车 a、卡车 b 或飞机 p 的控制最低成本限制,以及最短路径的限制。是否租赁卡车 a、b,货物是否通过航空运输到城市,0、1 分别选与不选卡车或飞机,定义决策变量用于表示:

$$x_i, y_i, z_i = \begin{cases} 1 & \text{选 } i \text{ 型卡车或飞机} \\ 0 & \text{不选 } i \text{ 型卡车或飞机} \end{cases} \quad (5.5)$$

并结合选择卡车 a 或卡车 b 的数量及其对应的成本与运输时间折现值,可建立卡车租赁成本、航空运输成本和罚函数项的目标函数:

$$\max Q = \sum_{i=1}^4 x_i y_i q_i \quad (5.6)$$

将其转化为最小化目标函数 $-Q$,即:

$$\min(-Q) = - \sum_{i=1}^4 x_i y_i q_i \quad (5.7)$$

约束条件由如下六点组成:

- 1.货物的起始地.目的地和数量是已知的且固定的,且货物必须从起点运送到终点。
- 2.每个城市可供租赁的卡车数量是足够的,但具体车型和数量需要根据成本和需求来选择。
- 3.卡车的运输时间和成本是给定的,需要在安排运输路线时考虑。
- 4.航空运输的运价是固定的,且当日到达,但需要权衡成本和时间因素。
- 5.各小组间可在任何一个城市拼货和中转运输,但需要合理安排以降低成本。
- 6.总成本需要在一定的预算范围内,或者需要尽可能地降低成本。

(1)运输工具的最大载重:

$$\begin{cases} \sum_{i=1}^n c_i x_i \leq 5000 i_+ \\ \sum_{i=1}^n c_i y_i \leq 3000 i_+ \\ \sum_{i=1}^n c_i z_i \leq 10000 p_i \end{cases} \quad (5.8)$$

(2)一次性运输货物时,卡车租赁总资金不能大于租赁飞机总资金的成本:

$$\sum_{i=1}^n n c_i x_i + \sum_{i=1}^n n c_i y_i \leq 10000 n p_i \quad (5.9)$$

(3)卡车 a, 卡车 b 与飞机 p 的数量必须为自然数:

$$x_i \in N \quad (5.10)$$

(4)由决策变量的定义可得:

$$y_i \in \{0,1\} \quad (5.11)$$

综上, 约束条件为:

$$s. t. \begin{cases} \sum_{i=1}^4 y_i \geq 3 \\ \sum_{i=1}^4 c_i x_i \leq 2400 \\ x_i \in N \\ y_i \in \{0,1\} \end{cases} \quad (5.12)$$

为了将上述二次有约束二进制优化模型改写为二次无约束二进制优化模型 (QUBO), 需要对目标函数添加惩罚:

$$g_1 = \left\{ \max \left| 3 - \sum_{i=1}^4 y_i, 0 \right| \right\}^2, k_1 = \left\{ \max \left| \sum_{i=1}^4 c_i x_i - 2400, 0 \right| \right\}^2 \quad (5.13)$$

故惩罚项为:

$$\lambda(g_1 + k_1) \quad (5.14)$$

其中 λ 为罚参数, 一个足够大的数。

因此最终 QUBO 模型为:

$$\min z = - \sum_{i=1}^4 x_i y_i q_i + \lambda(g_1 + k_1) \quad (5.15)$$

5.1.3 罚函数灵敏度分析

具体步骤:

1.将该模型的无约束优化问题表示为: $\min z(x_i, \lambda) = \min(-Q) + \lambda(g_1 + k_1)$

2.选择罚参数 λ , 设置一个基准值 λ_0

3.使用优化算法计算在该罚参数值 λ_0 下的目标函数值 $\min(-Q)$, 得到最优解 x_i^* 及对应目标函数值 $\min(-Q)^*$ 。

4.改变其罚参数值, 例如 $\lambda_{\alpha 0} = \lambda_0 + \Delta \varepsilon$, ε 是一个小的正数。

5.使用优化算法求解新的无约束优化问题: $\min z = \min(-Q) + \lambda_{\alpha 1}(g_1 + k_1)$, 其中 $\lambda_{\alpha 1}$ 为改变后的罚参数值, 得到新的最优解 x_i' 和对应新的目标函数值 $\min(-Q)'$ 。

6.计算灵敏度, 灵敏度可以通过计算罚函数值的变化量与罚参数的变化量之比来衡量, 即 $\frac{\min(-Q)^* - \min(-Q)'}{\lambda_0 - \lambda_{\alpha 1}}$

7.根据需要, 需要进行多次改变罚参数的值, 并计算相应的灵敏度, 以全面了解罚函数的特性。

这个灵敏度指标反映了罚函数对罚参数变化的敏感程度。例如, 如果灵敏度值较大, 说明罚函数对罚参数的变化较为敏感; 如果灵敏度值较小, 则说明罚函数对罚参数的变化不太敏感。

根据上述步骤求得 $\lambda=600000$ 时, 较为合适。

5.2 模型的求解

5.2.1 量子计算

量子退火算法是一类新的优化算法，它是模拟退火算法的一种延伸和改进。与经典模拟退火算法利用热力学中的热波动来搜索问题的最优解不同，量子退火算法利用量子隧穿效应即量子因为测不准原理而具有的穿透比其自身能量高的势垒的能力，从而使算法摆脱局部极值，以更高概率逼近全局最优值^[5]。如图 5.3 所示：

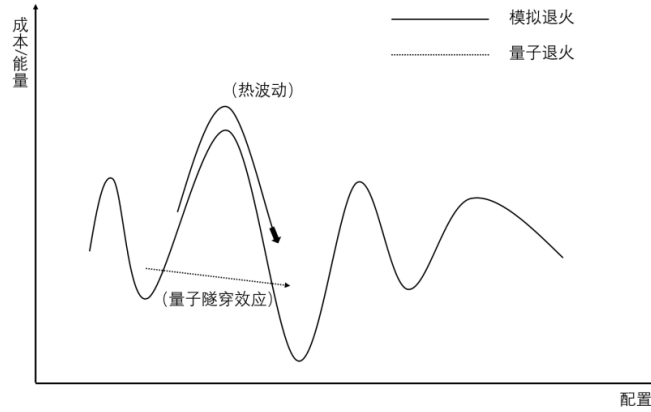


图 5.3 量子退火算法与模拟退火算法过程比较图

量子退火算法模型主要有两大核心要素。首先，是势能，它的主要作用在于实现量子优化问题与量子系统之间的映射关系。换言之，它我们将所追求的优化目标函数，转化为一个作用于量子系统的势场。其次，是动能，它通过引入一个可以调控的动能项，扮演了调节量子波动穿透场的角色。量子系统在这两个场的协同影响下的演化过程可以通过薛定谔方程（或狄拉克方程）来精确描述：

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = H(t) |\psi(t)\rangle \quad (5.16)$$

因实际情况下难以直接求解，故采用有效随机过程方法如路径积分蒙特卡罗 (PIMC):

$$E(t) = E_m(t) + E_n(t) \quad (5.17)$$

在量子退火算法中，量子哈密顿函数 $E(t)$ 作为评价函数，精准评估量子系统状态。其中， $E_m(t)$ 表示势能，反映能量分布； $E_n(t)$ 代表动能，引入量子波动效应，通过逐步减小初始值，引导系统逼近全局最优解。

量子退火算法常用横向场随机伊辛模型作为测试基准。诸多组合优化问题首先需映射至伊辛模型，随后运用量子退火算法求解。该模型的哈密顿函数是算法关键：

$$E_p = \sum_{a=1}^n e_a \delta_a^x + \sum_{a,b=1}^n I_{ab} \delta_a^x \delta_b^x \quad (5.18)$$

其中，能量偏移角度以 e_a 表示，泡利自旋矩阵以 δ_a^x 表示，自旋量 a 和 b 之间的耦合度以 I_{ab} 表示。

依据伊辛模型的哈密顿函数，从而获得量子退火的哈密顿函数为：

$$E(t) = E_p + \Gamma(t) \sum_{a=1}^n \Delta_a \delta_a^y \quad (5.19)$$

在量子退火算法中，场强 Γ 促使自旋状态发生转变，类似于模拟退火中温度 T 的作用。因此，任何采用上述形式表示的优化问题，均可借助量子退火算法得到解决。

5.2.2 算法设计与求解

QUBO 模型适配相干伊辛机并能转化为伊辛模型，因此设计量子计算算法，借助特定求解器如 Kaiwu SDK 进行求解，从而提高效率。

算法 1

Step1:根据最优租赁方案情境将其剖析为组合优化问题，构造函数。

Step2:初始化各个参数，其中计数参数 $i=1$ ，取量子退火的初始温度 T_0 足够大， Γ 为横向场强，确定最大迭代次数为 $MaxSteps$ ，初始化解为 x ，对应的状态能量为 $E_m(t)$ 。

Step3:通过随机将一个决策变量由 0 变 1 或由 1 变 0 产生随机微扰，从而产生新状态 x' ，其对应的状态能量为 $E_m(x')$ 。

Step4:计算能量差 $\Delta E_m = E_m(x') - E_m(t)$ 以及 $\Delta E_n = E_n(x') - E_n(t)$ ，如果 $\Delta E_m < 0$ 或者 $\Delta E_n < 0$ ，则系统接受新解，反之，计算 x' 的接受概率 $\exp(\Delta E_n/T)$ ，并产生 $(0,1)$ 区间上均匀分布的随机数 $random(0,1)$ 。若 $\exp(\Delta E_n/T) < random(0,1)$ ，则 $x = x'$ 。否则，重复执行步骤 3。

Step5:进行退温操作， Γ 的变化和模拟退火中的温度 T 作用类似，横向场强变化形式为 $\Gamma = \Gamma - (\Gamma_0/MaxSteps)$ 。

Step6:判断是否满足终止条件 $\Gamma=0$ ，如果满足，量子退火算法终止，否则，将重复步骤 3。

基于上述算法，使用 KaiwuSDK 内置的模拟退火求解器和 CIM 模拟器进行求解得，当租赁 1 辆卡车 a，2 辆卡车 b 时的总成本最小，为 58000 万元。

5.2.3 问题一的求解与最优方案

根据已知条件，可计算得出两种型号卡车的性价比大小关系为 $a > b$ 。

利用贪心算法求解进行检验。

首先，需先上文求解所得的结果相同，故结果合理，因此当租赁 1 辆卡车 a，2 辆卡车 b 时的总成本最小，为 58000 万元。

5.3 最优方案

5.3.1 货车租赁方案

表 6.8 问题一利用随机抽样的结果检测表

对比方法	货车租赁方案	最小总成本
量子退火算法	2 辆卡车 a	53600
	1 辆卡车 b	
随机抽样	2 辆卡车 a	78200
	1 辆卡车 b	

5.3.2 货物运输方案

表 6.9 问题一利用贪心算法求出最佳方案表

对比方法	货车租赁方案	最小总成本
量子退火算法	2 辆卡车 a	53600
	1 辆卡车 b	
随机抽样	2 辆卡车 a	78200
	1 辆卡车 b	

六、问题二模型的建立与求解

6.1 模型的建立

6.1.1 QUBO 模型的建立

问题二是在问题一的场景中，改变了组合租赁运输工具的各参数，可租赁卡车 a ， b 以及飞机 p 的各参数，以及两者之间的匹配关系和启动成本资金，而其问题本质与解题思路未发生改变，故问题二可在问题一已建立的模型基础上，修改改变后相对应的参数，进而建立模型。

运输成本的计算会受到货物重量和体积、运输距离、卡车类型和租金、卡车单趟成本、运输方式选择、油价波动、道路状况、运输时间、拼货和中转、市场供需关系、政策法规等多个因素的影响。题意中运输物流中投入使用的不同载量卡车型号 a ， b 数量 Y 为：

$$Y = \sum_{k=1}^a \sum_{i=1}^b w_{ki} t_{ki} (z_k \cdot d_k) w_{ki} t_1 P \quad (6.1)$$

物流运输的总工作量，可根据卡车的一天的 24 小时运输时间、卡车货箱容量、效率以及数量用数学表达式表示卡车的成本即：

$$C_1 = \sum_{k=1}^n \sum_{i=1}^{24} w_{ki} t_{ki} (s_k w_{ki} + y_k w_{ki} t_1 + r_k w_{ki} t_2 + p_k w_{ki} t_2) \quad (6.2)$$

转化为最小化目标函数，即：

$$C_2 = 7 \sum_{j=1}^2 (y'_j t_1 + r'_j t_2 + p'_j t_2) + 3(y'_3 t_1 + r'_3 t_2 + p'_3 t_2) \quad (6.3)$$

约束条件由如下三点组成：车辆载重约束确保每次运输不超过车辆的载重限制；时间窗口约束，考虑货物在运输所用的送达时间；供需平衡约束，确保每个城市的发货和收货量平衡。

6.1.2 罚函数灵敏度分析

具体步骤同问题一，改变了其目标函数与变量：这个灵敏度指标反映了罚函数对罚参数变化的敏感程度。例如，如果灵敏度值较大，说明罚函数对罚参数的变化较为敏感；如果灵敏度值较小，则说明罚函数对罚参数的变化不太敏感。

1. 将该模型的无约束优化问题表示为： $\min z(x_i, \lambda) = \min(-Q) + \lambda(g_1 + k_1)$

2. 选择罚参数 λ ，设置一个基准值 λ_0

3. 使用优化算法计算在该罚参数值 λ_0 下的目标函数值 $\min(-Q)$ ，得到最优解 x_i^* 及对应目标函数值 $\min(-Q)^*$ 。

4. 改变其罚参数值，例如 $\lambda_{\alpha 0} = \lambda_0 + \Delta \varepsilon$ ， ε 是一个小的正数。

5. 使用优化算法求解新的无约束优化问题： $\min z = \min(-Q) + \lambda_{\alpha 1}(g_1 + k_1)$ ，其中 $\lambda_{\alpha 1}$ 为改变后的罚参数值，得到新的最优解 x_i' 和对应新的目标函数值 $\min(-Q)'$ 。

6. 计算灵敏度，灵敏度可以通过计算罚函数值的变化量与罚参数的变化量之比来衡量，即 $\frac{\min(-Q)^* - \min(-Q)'}{\lambda_0 - \lambda_{\alpha 1}}$

7. 根据需要，需要进行多次改变罚参数的值，并计算相应的灵敏度，以全面了解罚函数的特性。

由上述步骤求解得： $\lambda = 600000$ 时，较为合适。

6.1.3 物流转运中心选址

利用数学规划方法基于数学规划的物流中心选址方法能够系统地处理大量的数据和复杂的约束条件，要综合考虑成本、服务水平、地理位置等多重因素，通过线性规划、非线性规划、整数规划等，针对不同的选址问题设定适当的目标函数和约束条件，从而找到最优或接近最优的选址方案。

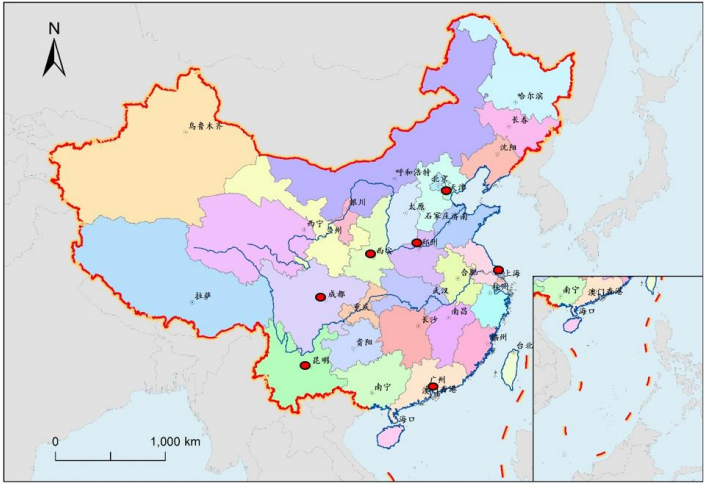


图 6.1 物流配送选址点图

结合 GIS 提供精准地理数据，MCDM 平衡多元目标，数学规划方法显著增强物流选址决策的科学性与可靠性，助力企业精准布局，应对复杂市场环境，实现资源最优配置与成本有效控制。

表 6.2 公司一待运货物小组 1

货物类型	起点城市	终点城市	卡车 a	卡车 b	运输时间（天）	成本	飞机
普货（19 吨）	上海	天津	1	1	3	28500	0
普货（25 吨）	西安	昆明	2	1	4	53000	0
普货（7 吨）	郑州	深圳	2	0	4	28000	0

在此问题中，两物流公司计划在成都、上海、天津物流起点到终点西安、昆明、深圳在运输区域内建立一个新的物流转运中心，根据这些数据如图 6.2 所示，基于重心法数学模型，可找到一个地理位置，以最小化从始发物流中心到目的城市的总运输成本和时间。

表 6.3 公司一待运货物小组 2

货物类型	起点城市	终点城市	卡车 a	卡车 b	运输时间（天）	成本	飞机
普货（27 吨）	上海	天津	2	1	3	38000	0
普货（10 吨）	成都	昆明	2	0	4	32000	0
普货（19 吨）	上海	深圳	1	2	4	44500	0

现有两公司的配送节点信息，6 个配送节点的 x、y 坐标，需求量和运输费用，如表 6.4 所示。

表 6.4 原始坐标结果

节点	wX9i	wY9i	wZ9i
天津	300	800	100
昆明	1130	300	150
上海	375	937.5	187.5
西安	450	275	72
郑州	900	900	112.5
合计	3155	3212.5	622
x1=	5.16	y1=	5.18

6.1.4 改进麻雀优化算法

1. 麻雀算法基本原理

麻雀搜索算法(SparrowSearchAlgorithm, SSA)是由薛建凯提出的一种新型群智能优化算法, 主要模拟了麻雀群体觅食和反捕食行为^[6]。

每代发现者的位置更新公式如下:

$$x_{i,d}^{t+1} = \begin{cases} x_{i,d}^t \times \exp\left\{\frac{-i}{\alpha \times iter_{max}}\right\}, & \text{if } R_2 < ST \\ x_{i,d}^t + Q \times L, & \text{if } R_2 \gg ST \end{cases} \quad (6.4)$$

每代跟随者的位置更新公式如下:

$$x_{i,d}^{t+1} = \begin{cases} Q \times \exp\left\{\frac{xw_d^t - x_{i,d}^t}{t^2}\right\}, & \text{if } i > \frac{N}{2} \\ xb_d^t + \frac{1}{D} \sum_{d=1}^D (|x_{i,d}^t - xb_d^t| \times rand\{-1,1\}), & \text{else} \end{cases} \quad (6.5)$$

侦查者位置更新公式如下:

$$x_{i,d}^{t+1} = \begin{cases} xb_d^t + \beta \times |x_{i,d}^t - xb_d^t|, & \text{if } f_i \neq f_g \\ x_{i,d}^t + K \times \left\{\frac{|x_{i,d}^t - xw_d^t|}{|f_i - f_w| + \varepsilon}\right\}, & \text{if } f_i = f_g \end{cases} \quad (6.6)$$

2. PSO 算法算最短路径

粒子群优化算法(Particle Swarm Optimization, PSO 主要通过更新粒子的速度和位置信息寻找最优解。粒子群优化算法的应用较为广泛, 且收敛速度快, 调整参数也较少^[7]。考虑有 N 个粒子在 D 维空间搜索, 初始粒子群算法更新表示为:

$$\begin{cases} v_{ij}^{d+1} = \omega v_{ij}^d + c_1 r_1 (p_{ij}^d - x_{ij}^d) + c_2 r_2 (p_{gj}^d - x_{ij}^d) \\ v_{ij}^{d+1} = v_{ij}^d + v_{ij}^{d+1} \end{cases} \quad (6.7)$$

其中, d 为迭代次数, v_{ij} 表示第 i 个粒子在 i 维的速度, x_{ij} 表示第 i 个粒子在 j 维的位置。ω 表示惯性权重, 控制粒子速度的惯性。 c_1 与 c_2 是学习因子, 控制粒子个体和群体经验对速度的影响。 r_1 与 r_2 是范围在 $[0, 1]$ 之间的随机数。

3. 麻雀算法改进

①Cubic 混沌映射初始化麻雀种群。在优化领域, 混沌映射可以用于替代伪随机数生成器, 生成 0 到 1 之间的混沌数。Cubic 混沌映射具有良好的混沌特性, 能够在整个搜索空间内生成分布广泛的初始解, 从而提高算法的全局搜索能力。

Cubic 表达式如下:

$$x_{n+1} = \alpha x_n (1 - x_n^2) \quad (6.8)$$

其中, x_n 是当前迭代的混沌变量值, 初值为(0, 1), α 是系统参数, 它决定了映射的混沌行为, 取 $x_n=0.2$, $\alpha=0.2493$ 。

②重心反向学习。反向学习是一种经典的智能优化算法加速技术, 它在当前点和它的反向点之中择优选择。

重心计算公式如下:

$$M = \frac{1}{n} \sum_{i=1}^n x_i \quad (6.9)$$

重心反向解的计算公式:

$$x_{i,new} = 2 \times k \times M - x_i \quad (6.10)$$

在迭代过程中, 算法会对麻雀种群的更新位置进行重心反向变异。由于不能保证每次变异都能得到更优的位置, 因此采用贪心策略来决定是否更新位置: 仅当变异后的新位置更优时, 才用其替换原有位置, 否则保留原位置。其中, k 是 $[0, 1]$ 范围内均匀分布的随机数, 加入收缩因子可以拓展反向搜索空间的范围, 增大找到更优解的概率。

4. 混合算法求解过程

混合麻雀算法和粒子群算法的核心思想在于, 将粒子群算法的位置信息作为改进后麻雀算法的相关参数, 从而运行麻雀算法, 最终进行模型求解, 如图所示。

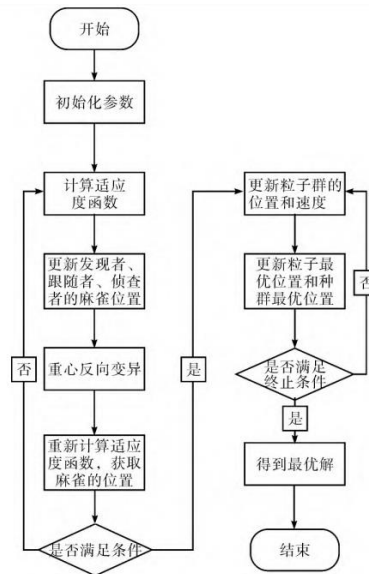


图 6.2 算法求解过程

混合算法的运行步骤如下:

混合算法

步骤 1 初始化: 设定麻雀和粒子群的种群数量、迭代次数等参数。利用公式(6.8)初始化麻雀种群位置, 增加搜索范围的多样性。

步骤 2 适应度评估与排序: 根据适应度函数对麻雀进行排序, 识别并记录每个麻雀的个体最优和全局最优适应度值及对应位置。

步骤 3 位置更新: 使用公式(6.4)、公式(6.5)、公式(6.6)更新发现者、跟随者、侦察者的位置。

步骤 4 重心反向变异: 对处于最优位置的麻雀使用公式(6.10)实施重心反向变异, 增强全局搜索能力。

步骤 5 适应度再评估: 重新计算适应度值, 更新麻雀的个体和全局最优位置。

步骤 6 位置优化判断: 比较麻雀的当前最优位置与粒子群的最优位置。如果麻雀位置更优, 进入粒子群优化阶段; 否则, 回到步骤 2 继续迭代。

步骤 7 粒子群位置与速度更新: 使用公式(6.7)更新各粒子群的位置和速度。

步骤 8 粒子群最优位置更新: 更新粒子群个体最优和全局最优位置。

步骤 9 迭代终止判断: 如果迭代次数达到预设的最大值, 则结束, 输出全局最佳位置; 这一位置作为麻雀算法的重要参数, 用于求解目标函数。如果未达到最大迭代次数, 回到步骤 6 继续迭代。

为了验证所提算法的优越性, 本算法选取公司 1 的物流配送和 10 个主要地级市物流中心为例进行路径规划。配送中心和地级市物流中心的位置分布如图 6.3 所示。

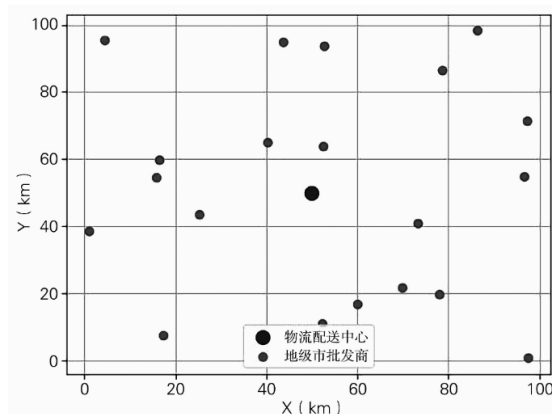


图 6.3 物流配送中心和地级市批发商分布图

6.2 模型的求解

6.2.1 subQUBO 模型的求解

SubQUBO 方法的核心思想是将大规模的组合优化问题分解为一系列较小规模的问题，以便利用现有的计算资源，如伊辛机，进行有效求解，并通过理论支持，确保分解后的小问题在求解后能够还原到原始大规模问题的解^[8]。

实现逻辑：化整为零，反向求解

1. 在解决大规模组合优化问题时，QUBO 建模的解往往由多个量子比特集合构成。

2. 若我们构建一个较小规模的问题，专注于最终解中所有不正确的量子比特集合，并利用伊辛机进行求解，那么这些错误的量子比特集合将被修正为正确的集合，从而得出正确的解。

实现方法：设计问题，聚焦于错误的量子比特集合，并通过伊辛机多次求解。

1. 使用经典计算器生成多个候选答案，比较它们的量子比特集合值。一致的集合视为正确，不一致的视为错误。

2. 提取错误集合，利用伊辛机求解，最终得到整体的正确答案。

6.2.2 罚函数灵敏度分析

本题根据两个决策变量形成的 QUBO 模型，已知考虑的约束条件过多，因此设计量子退火算法以降低计算量。

算法 3

Step1: 根据最优租赁方案情境将其剖析为组合优化问题，构造函数。

Step2: 初始化各个参数，其中计数参数 $i=1$ ，取量子退火的初始温度 T_0 足够大， Γ 为横向场强，确定最大迭代次数为 $MaxSteps$ ，初始化解为 x ，对应的状态能量为 $E_m(t)$ 。

Step3: 通过随机将一个决策变量由 0 变 1 或由 1 变 0 产生随机微扰，从而产生新状态 x' ，其对应的状态能量为 $E_m(x')$ 。

Step4: 计算能量差 $\Delta E_m = E_m(x') - E_m(t)$ 以及 $\Delta E_n = E_n(x') - E_n(t)$ ，如果 $\Delta E_m < 0$ 或者 $\Delta E_n < 0$ ，则系统接受新解，反之，计算 x' 的接受概率 $\exp(\Delta E_n/T)$ ，并产生 $(0,1)$ 区间上均匀分布的随机数 $random(0,1)$ 。若 $\exp(\Delta E_n/T) < random(0,1)$ ，则 $x = x'$ 。否则，重复执行步骤 3。

Step5: 进行退温操作， Γ 的变化和模拟退火中的温度 T 作用类似，横向场强变化形式为 $\Gamma = \Gamma - (\Gamma_0/MaxSteps)$ 。

Step6: 判断是否满足终止条件 $\Gamma=0$ ，如果满足，量子退火算法终止，否则，将重复步骤 3 对下一个小问题重复 Step2-Step5。否则，重复 Step3。当所有小问题算法终止，整个量子退火算法终止。

使用 Kaiwu SDK 求解得：小组一的货物有从上海到西安的 19 吨普货，成本 24500 元；从成都到昆明的 25 吨普货，成本 37000 元；从上海到深圳的 7 吨普货，成本 14000

元。小组二的货物有从上海到西安的 27 吨普货，成本 38000 元；从成都到昆明的 10 吨普货，成本 16000 元；从上海到深圳的 19 吨普货，成本 30500 元。

6.2.3 物流转运中心选取

物流配送路径优化问题，提出一种改进的麻雀搜索算法，通过引入 Cubic 混沌映射和粒子群优化技术，有效地提高了算法的全局搜索能力，同时有效避免了早熟收敛的问题。

表 6.5 初始化迭代的运输距离和总运输成本

节点	坐标 x	坐标 y	w 需求量/吨	q 运输成本/(元/吨.)	K	d 运输距离	TC 运输成本
昆明	118.77	32.04	46	2510.869565	10	2552	115500
上海	111.67	40.82	35	2928.571429	10	1474	102500
西安	117.19	39.13	44	2443.181818	10	915	107500
天津	102.71	25.04	27	3222.222222	10	2087	87000
郑州	108.95	34.26	30	3250	10	889	97500
总运输成本							510000

由于第一次迭代结果的总运输成本小于初始运输成本，因此需要进行多次迭代，直至总运输成本高于上一次总运输成本，即可得出最优物流中心坐标。多次迭代结果数据如表 6.6 所示。

表 6.6 迭代最终结果的物流中心坐标

节点	wX9i	wy9i	w9;
郑州	300	700	100
西安	1130	346	146
上海	375	937.5	187.5
昆明	450	275	72
天津	895.5	600	112.5
合计	3150.5	2858.5	618
x1=	5.61	y1=	5.12

在实验中，将上述参数和数据输入模型，并通过 Python 实现了相关算法，模型得出的车辆配送路径结果如图 6.4 所示

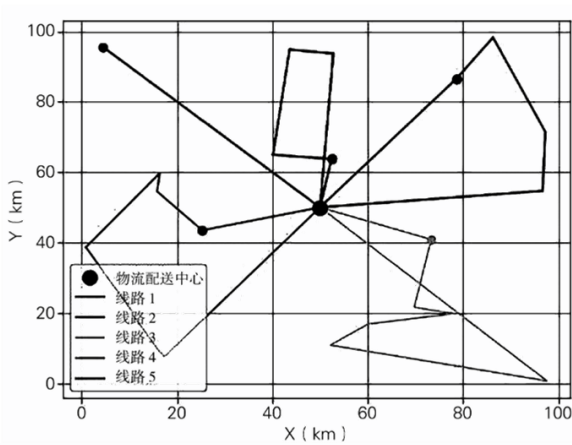


图 6.4 车辆配送路径图

在坐标轴上（5.61,5.12）这一点恰好位于西安与郑州之间的中心地带，使其成为设立转运中心的绝佳位置。此点不仅便于货物从六个主要城市快速集散，还能通过高效的物流网络辐射全国。选择此处作为转运中心，能显著提升物流效率，降低运输成本。我们建议在该区域构建一座集现代化设施与智能管理于一体的转运枢纽，以实现物流

资源的优化配置和快速响应市场需求^[9]。

6.2.4 最优化成本

成本最优化策略重点关注降低整体运营成本，以确保高效的物流服务。基于数学规划方法，可以评估不同地点的成本效益比，包括地块价格、运输成本（如距离主要公路、铁路线的远近），以及地理位置所覆盖的服务范围等。对物流中心的选择可以在运输成本和地理位置之间找到一个平衡点。

另外，成本最优化策略还涉及物流中心的设计和运营，可以通过优化货物入库、存储、拣选和出库等仓库内部流程，提高运营效率，减少时间和成本，同时也可以通过采用高效的货物追踪系统等现代化的物流设施来节约运营成本。可见，基于数据和模型的方法能够为物流中心的选址和运营提供更精准的指导，使物流中心在保持竞争力的同时，实现成本最小化。

采用随机抽样方法，在满足约束条件，并采用最优匹配关系时的租赁方案中随机抽取部分方案，将其总成本与量子退火算法求得的总成本进行比较，结果如下表：

表 6.9 问题二利用随机抽样的结果检测表

对比方法	货车租赁方案	最小总成本
量子退火算法	1 辆卡车 a	53642
	2 辆卡车 b	
随机抽样	2 辆卡车 a	71542
	1 辆卡车 b	

表 6.10 改进麻雀算法及其他算法物流路径优化结果

名称	改进麻雀算法	SSA-PSO	SSA	PSO
总运输距离(km)	2070.8	1481.7	582.6	523.1
总费用(元)	28500	11800	7550	5850
使用车辆数(辆)	5	5	4	2
运行时间(s)	23.26	21.32	19.57	13.13

6.3 复杂组合成本最优化方案

6.3.1 货车租赁方案

在利用 subQUBO 方法和模拟退火算法解决复杂组合下的卡车租赁问题时，我们根据附件 4 中的详细指导和编程环境进行了相应的设置与调试。具体而言，我们设定了初始温度为 100 度，衰减常数为 0.95，并设定了迭代 25 次作为停止准则。通过这一系列的参数配置，我们成功运行了模拟退火算法，并得出了 subQUBO 模拟退火的结果图。

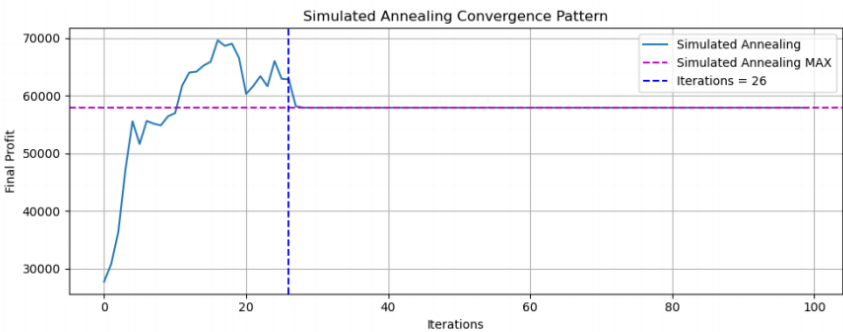


图 6.5 模拟退火求解结果

在结果图中，我们可以清晰地观察到随着迭代次数的增加，成本逐渐降低并趋于稳定，最终达到了模拟退火算法下的最小成本点。这一结果为我们提供了在复杂组合条件下卡车租赁方案的重要参考。

表 6.11 问题二租赁卡车数量及其货物重量的匹配关系表

起点-终点	货物类型	货物重量（吨）	成本（元）
上海-西安	普货	19	24,500
成都-昆明	普货	25	37,000
上海-深圳	普货	7	14,000
上海-西安	普货	27	38,000
成都-昆明	普货	10	16,000
上海-深圳	普货	19	30,500

基于模拟退火算法的输出，我们可以得出以下结论：在给定的问题背景下，通过合理安排不同租赁期限的卡车，两个物流公司合作运输最小成本的货车租赁方案如表 6.11 所示。

表 6.11 两公司合作运输最小成本的货车租赁表

货物地点	货物重量（吨）	卡车组合	A 型趟数	B 型趟数	总成本（元）
西安	44	A+B	3	1	78,000
		仅 A	4	0	94,000
昆明	46	A+B	3	2	97,500
		仅 A	4	0	100,000
上海	33	A+B	2	1	42,500
		仅 A	3	0	54,000

6.3.2 货物运输方案

本表格汇总了不同货物地点、两公司合作后货物总重量及卡车不同类型下的运输成本。详细列出了每趟运输的固定成本与租赁成本，进而计算出总运输成本与总租赁成本。结合所需趟数与预计总时间，最终得出总成本，如表 6.12 所示。此表格为物流规划与成本控制提供了直观的数据支持，帮助决策者快速比较不同运输方案的经济性，选择最优方案以节省成本。

表 6.12 最佳货物运输方案

货物地点	两公司总货物重量（吨）	卡车类型	所需趟数	每趟运输成本（元）	总运输成本（元）	每趟租赁成本（元）	总成本（元）
西安	44	A (12 吨)	4	18,500	74,000	5,000	94,000
		B (5 吨)	9	16,000	144,000	3,000	160,200
昆明	46	A (12 吨)	4	20,000	80,000	5,000	101,600
		B (5 吨)	10	17,000	170,000	3,000	200,000
上海	33	A (12 吨)	3	13,000	39,000	5,000	50,700
		B (5 吨)	7	11,000	77,000	3,000	108,700

七、问题三模型的建立与求解

7.1 人工智能模型超参数调优的 QUBO 模型

7.1.1 人工智能超参调优应用

QUBO 模型通常用于解决组合优化问题，其中变量是二进制(取值为 0 或 1)，并且目标函数是二次型的形式。它适用于那些需要在给定的约束条件下寻找最优解的问题，特别是当问题可以转化为进制变量和二次型目标函数的形式时。这种模型在多个领域都有应用，包括但不限于：

1.电路设计：QUBO 模型可以用于优化逻辑门的布局，以最大程度减少电路中的延迟或功耗。

2.物流和运输：用于优化货物的配送路线，以最小化成本或最大化效率。

3.金融投资：用于优化投资组合，以最大化收益或最小化风险。

4.组合优化：解决如旅行商问题(TSP)和背包问题等的组合优化问题。

5.人工智能和量子计算：QUBO 模型与量子计算和量子优化密切相关，可用于解决在传统计算机上难以解决的问题。

在问题 3 中，我们需要举例一个潜在通过构建 QUBO 模型从而进行决策优化的应用场景，综上并进行团队探讨，我们选取“人工智能模型超参数调优”这一场景进行探究。

7.1.2 背景信息

随着人工智能向分布式、多专家协同系统发展，机器学习模型的超参数调优变得尤为关键且复杂。传统调优方法如贪心算法、网格搜索，面对高维、大规模数据集时效率低下。

量子计算以其独特的并行处理能力和对特定问题的高效解决能力，为超参数调优带来了革命性机遇。尽管量子技术尚处初期，但其潜力巨大，能显著加速复杂模型超参数的搜索过程，提升模型性能调优的效率与精确度。探索量子计算在人工智能超参数优化中的应用，不仅是科研的前沿课题，更是推动 AI 技术跃升的重要方向。

7.1.3 研究方法

我们希望通过量子计算的优势，加速人工智能模型的超参数调优过程。具体而言，我们将超参数作为决策变量，以模型的性能度量（例如准确度）作为目标函数。QUBO 模型构建：我们将每个超参数引入二进制变量，构建一个 QUBO 模型。

基于超参数之间的相互关系，目标函数旨在最大化或最小化性能度量。CoherentIsingMachines(CIM)的应用：利用 CIM 的并行计算和高度连接性，在量子计算中更有效地搜索超参数空间。CIM 的量子优势可以提高搜索效率，找到更优的超参数组合。

7.1.4 思路及技术路线

选择关键超参数，我们需要仔细选择对模型性能影响较大的关键超参数。这可能包括学习率、层数、节点数等，具体取决于所使用的机器学习模型和任务。

7.1.5 QUBO 模型设计

1、决策变量的引入：对于每个选择的超参数，引入一个二进制变量表示其取值。

2、目标函数的设计：设计目标函数，以模型的性能度量为目标，最大化或最小化这一性能度量，形成形式为二次型目标函数的。然后探究函数的约束条件，引入惩罚因子，使目标函数不受约束条件影响，形成二元无约束二值函数。

模拟退火求解或 CIM 模拟求解：使用模拟退火算法或者 CIM 求解构建好的 QUBO 模型。其中，它们充分利用了量子计算的优势，能够在高维、复杂的问题中进行高效的求解。使用获得的最优超参数组合训练机器学习模型，并在验证集或测试集上评估其性能。比较使用量子计算方法和传统方法得到的超参数组合的性能。

通过这一技术路线，我们期望在人工智能模型的超参数调优中充分发挥量子计算的优势，提高优化过程的效率，以更好地支持人工智能领域的研究和应用。因此，根据人工智能超参数调优的 QUBO 模型绘制出技术路线图，如图 7.1 所示。

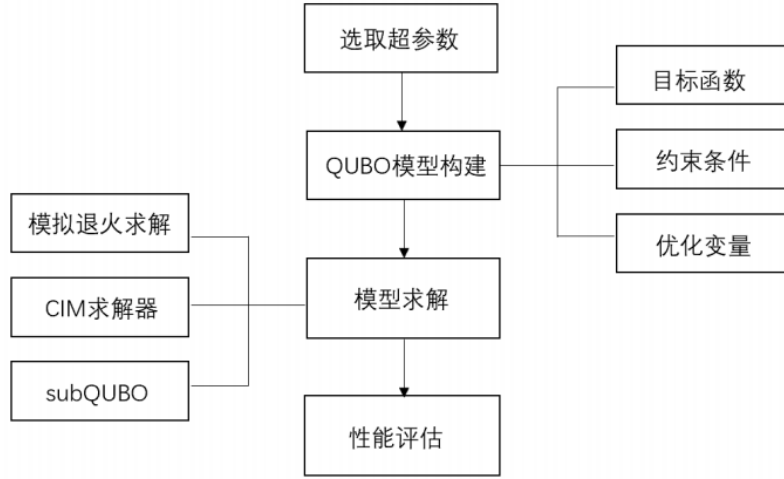


图 7.1 技术路线

7.2 研究结果

通过将量子计算引入人工智能模型的超参数调优过程，我们期望实现以下目标：

1.更快的优化速度：量子优化算法具有并行性和高效性，可以加速超参数搜索的过程，从而减少了调优所需的时间。

2.更准确的超参数调整：量子优化算法可能能够更精确地找到超参数的最优解，从而提高了模型的性能和泛化能力。

3.解决高维空间问题：传统的优化算法在处理高维超参数空间时可能受限，而量子优化算法可能更适用于处理高维空间的优化问题，这对于复杂的人工智能模型特别有用。

4.克服局部最优解问题：量子优化算法通常具有更好的全局搜索能力，可以帮助模型避免陷入传统算法局部最优解的情况，从而提高了模型的性能和稳定性。

5.应用于多种人工智能模型：量子优化算法不仅适用于特定类型的模型，还可以应用于各种类型的人工智能模型，包括监督学习、无监督学习、强化学习等。

7.2.1 QUBO 模型算法

QUBO 模型算法步骤

Step1:根据待优化问题，构造量子系统的评价函数 $H_q = H_{pot} + H_{kin}$,即量子哈密顿函数。其

中， $H_{pot}(t)$ 为势能，即模拟退火算法中的评价函数， $H_{kin}^{(t)}$ 为动能；

Step2:初始化各个参数， T_0 为量子退火的初始温度， Γ 为横向场强，变化的横向场强引起不同量子状态之间的量子跃迁，最大迭代次数为 MaxSteps，初始化状态为 x ,对应的状态能量为 $H_{pot}(x)$;

Step3:随机微扰产生新状态 x' ,对应的状态能量为 $H_{pot}(x')$;

Step4:计算能量差 $\Delta H_{pot} = H_{pot}(x') - H_{pot}(x)$ 以及 $\Delta H_q = H_s(x') - H_q^{(x)}$,如果 $\Delta H_{pot} < 0$ 或者 $\Delta H_q < 0$ 则系统接受新解 $x = x'$,反之如果 $\exp(\Delta H_q/T) < \text{randon}(0,1)$,则 $x = x'$,否则重复执行步骤 3;

Step5:进行退温操作， Γ 的变化和模拟退火中的温度 T 作用类似，横向场强变化形式为 $\Gamma = \Gamma - (\Gamma_0/\text{MaxSteps})$;

Step6:判断是否满足终止条件 $F = 0$,如果满足量子退火算法终止，否则重复步骤 3。

7.2.2 超参数调优 QUBO 模型

通过数据的获得，我们采用量子计算的方法对人工智能超参数进行初步探究。假设 Ni: 量子位的数量；Ne: 能量水平；Ns: 可解的个数，我们求解得到了 QUBO 求解

算法在不同参数配置下的准确度分布情况，例如：Ni=20，Ne=5 和 Ns=10。

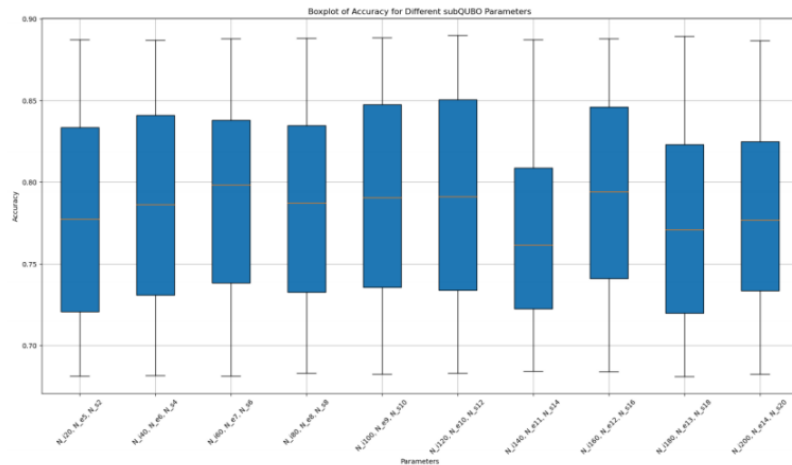


图 7.2QUBO 准确度分布箱线图

上图为箱形图，展示了 QUBO 求解算法在不同参数配置下的准确度分布情况。其中，每个箱子代表一组特定参数下算法准确度的统计分布。箱形图通过展示数据的五数概括（最小值、第一四分位数、中位数、第三四分位数、最大值）以及可能的异常值来展示数据的分散情况。

我们发现，准确率的中位数在不同参数设置之间有所变动，但总体来说几乎都在 0.80 以上，准确率较高。四分位距代表准确率的可变性，较短的箱子表示更一致的结果，较长的箱子表示结果的变化较大。结果显示 Ni₆₀,Ne₇,Ns₆，有些参数设置的箱子较长，表明在这些设置下准确率的变动较大，而一些较短的箱子则表明在这些设置下准确率较为稳定。

七、模型的评价与推广

7.1 模型的优点

本文通过模拟退火算法为 QUBO 问题得到了满意的解，较好地解决了寻找最优解的问题。且该算法还可用于其它各种组合优化问题，如 TSP 和 Knapsack 问题等，能产生令人满意的近似最优解，而且运行速度较快。

该模型通过两种求解方法分别是模拟退火求解器和 CIM 模拟器进行求解，其结果更合理、更准确；当量子比特的数量较多时，采用改进的模拟退火 subQUBO 进行求解，能很好地解决大规模数据或运算较复杂的问题。

本文对求解结果进行可视化处理及误差可视化处理，使呈现更为直观，且增强了可信度。

本文通过 QUBO 转化，将决策变量转化为 0-1 变量，使得数据得到充分使用，避免了数据错误或遗漏带来的误差。

7.2 模型的缺点

该模型建立在卡车租赁成本确定的前提条件下，本文采用的模拟退火算法求解 QUBO 问题时更适用于数据量足够大的样本，如果数据量较小，求解结果会存在一定的误差。

7.3 模型的推广

该论文使用的 QUBO 模型，除了可解决物流租赁和运输方案外的问题，还可以解决金融、人工智能、最大独立集问题、不对称分配问题、对称分配问题、边约束分配问题、二次背包问题、最大团问题、最大割问题、整数划分问题和旅行商问题等，可结合具体实际运用进行变化运用。并在使用过程中，可以通过下述方式将 QUBO 模型与 Ising 模型相互转化，以便求解不同问题。

参考文献

- [1] Wang Xiaojun, Wang Zhenghuan, Ni Bowen. Mapping structural topology optimization problems to quantum annealing [J]. Structural and Multidisciplinary Optimization, 2024, 67 (5):
- [2] 王敬,张萌,李芳,张海懿.量子计算关键技术及应用发展分析[J].信息通信技术与政策,2023,49(07):9-16.
- [3] Şeker Oylum, Tanoumand Neda, Bodur Merve. Digital Annealer for quadratic unconstrained binary optimization: A comparative performance analysis [J]. Applied Soft Computing Journal, 2022, 127
- [4] Yue Ji. Logistics distribution scheduling algorithm based on artificial intelligence [J]. Measurement: Sensors, 2024, 34
- [5] 王宝楠,水恒华,王苏敏,胡风,王潮.量子退火理论及其应用综述[J].中国科学:物理学力学天文学,2021,51(08):5-17.
- [6] 孙玉杰,张占强,孟克其劳,吕晓圆.基于 ESMD 和 SSA-PNN 的电能质量扰动信号识别分类[J].现代电子技术,2022,45(14):108-114.
- [7] 曹梦龙,赵文彬,陈志强.融合粒子群算法与改进灰狼算法的机器人路径规划[J].系统仿真学报,2023,35(08):1768-1775.
- [8] Luca Asproni, Davide Caputo, Blanca Silva, Giovanni Fazzi, Marco Magagnini. Accuracy and minor embedding in subqubo decomposition with fully connected large problems: a case study about the number partitioning problem [J]. Quantum Machine Intelligence, 2020, 2 (1):
- [9] 吴善兵.六西格玛方法在 QG 公司厂内物流管理改善中的应用研究[D].导师:关志民.东北大学,2015.

附录

```
1. # -----导入模块-----
2. import kaiwu as kw
36.     laps = 50,
37.     dt = 0.1,
38.     normalization = 0.3,
39.     iterations = 10)
40. opt = kw.sampler.optimal_sampler(matrix, output, 0)
41. best = opt[0][0]
42. max_cut = (np.sum(-matrix)-np.dot(-matrix,best).dot(best))/4
43. print("The obtained max cut is "+str(max_cut)+".")

问题一对比线性规划
44. from pulp import LpMaximize, LpProblem, LpVariable, LpBinary, LpInteger
45.
46. # 创建问题实例
47. model = LpProblem(name="resource-allocation", sense=LpMaximize)
48.
49. # 定义整数决策变量
50. w1 = LpVariable(name="w1", lowBound=0, cat=LpInteger)
51. w2 = LpVariable(name="w2", lowBound=0, cat=LpInteger)
52. w3 = LpVariable(name="w3", lowBound=0, cat=LpInteger)
53. w4 = LpVariable(name="w4", lowBound=0, cat=LpInteger)
54.
55. # 引入二进制变量
56. y1 = LpVariable(name="y1", cat=LpBinary)
57. y2 = LpVariable(name="y2", cat=LpBinary)
58. y3 = LpVariable(name="y3", cat=LpBinary)
59. y4 = LpVariable(name="y4", cat=LpBinary)
60.
61. # 目标函数
62. model += (2000 * w1 + 3000 * w2 + 5000 * w3 + 6000 * w4, "Total Profit")
63.
64. # 约束条件
65. model += (100 * w1 + 140 * w2 + 200 * w3 + 320 * w4 <= 2400, "Resource
Limit")
66.
67. # 二进制变量约束，确保至少 3 个非零
68. epsilon = 1 # 对于整数变量，最小非零值是 1
69. model += (y1 + y2 + y3 + y4 >= 3, "At Least Three Nonzero")
70. model += (w1 >= epsilon * y1, "Link w1 and y1")
71. model += (w2 >= epsilon * y2, "Link w2 and y2")
72. model += (w3 >= epsilon * y3, "Link w3 and y3")
73. model += (w4 >= epsilon * y4, "Link w4 and y4")
74.
75. # 求解问题
76. model.solve()
```

```

77.
78. # 输出结果
79. print("Status:", model.status) # 输出模型的求解状态
80. print("Objective Value:", model.objective.value()) # 输出目标函数的最大值
81. print("Values of Variables:")
82. print("w1 =", w1.value())
83. print("w2 =", w2.value())
84. print("w3 =", w3.value())
85. print("w4 =", w4.value())
86.
87. 问题一 QUBO
88. import pulp as lp
89.
90. # 初始化问题
91. model = lp.LpProblem("Optimized_Mining_Equipment_Purchase_and_Matching",
lp.LpMaximize)
92.
93. # 决策变量：普卡租赁数量
94. x = lp.LpVariable.dicts("Excavators", [1, 2, 3, 4], lowBound=0,
cat='Integer')
95.
96. # 普卡参数
97. params = {
98.     1: {'efficiency': 190, 'fuel': 28, 'price': 100, 'labor': 7000,
'maintenance': 1000},
99.     2: {'efficiency': 175, 'fuel': 30, 'price': 140, 'labor': 7500,
'maintenance': 1500},
100.    3: {'efficiency': 165, 'fuel': 34, 'price': 200, 'labor': 8500,
'maintenance': 2000},
101.    4: {'efficiency': 150, 'fuel': 38, 'price': 320, 'labor': 9000,
'maintenance': 3000}
102. }
103.
104. # 卡车数量
105. trucks = {1: 7, 2: 7, 3: 3}
106.
107. # 每月工作天数和每天工作小时数
108. days_per_month = 20
109. hours_per_day = 8
110. fuel_price = 7
111. ore_price = 20
112.
113. # 成本和收益计算
114. total_initial_cost = lp.lpSum(x[i] * (params[i]['price'] * 10000)
for i in params)
115. total_operating_costs = lp.lpSum(x[i] * (
116.     (params[i]['fuel'] * hours_per_day * days_per_month * fuel_price)
+
117.     (params[i]['labor'] + params[i]['maintenance']) * days_per_month

```

```

118. ) for i in params) * 12
119. total_revenue = lp.lpSum(x[i] * params[i]['efficiency'] *
hours_per_day * days_per_month * ore_price
120.         for i in params) * 12
121.
122. # 目标函数
123. model += (total_revenue - total_operating_costs - total_initial_cost)
124.
125. # 添加卡车数量限制和效率调整
126. for i in params:
127.     if i == 1:
128.         model += x[1] <= trucks[1] # 普卡 1 匹配卡车 1
129.     elif i == 2:
130.         model += 2 * x[2] <= trucks[1] + trucks[2] # 普卡 2 匹配货
车 1 或卡车 2
131.     elif i == 3:
132.         model += 2 * x[3] <= 2 * trucks[1] + 2 * trucks[2] +
trucks[3] # 普卡 3 匹配卡车 1, 2 或 3
133.     elif i == 4:
134.         model += 2 * x[4] <= 2 * trucks[2] + trucks[3] # 普卡 4 匹
配卡车 2 或 3
135.
136. # 确保总启动成本不超过 2400 万元
137. model += total_initial_cost <= 2400 * 10000
138.
139. # 求解问题
140. model.solve()
141.
142. # 输出结果
143. print("Status:", lp.LpStatus[model.status])
144. for i in params:
145.     print(f"Number of Excavator {i} purchased:", x[i].value())
146.
147. # 输出经济指标
148. print("Total Initial Cost: ", total_initial_cost.value())
149. print("Total Operating Costs: ", total_operating_costs.value())
150. print("Total Revenue: ", total_revenue.value())
151.
152. 转化为将之前计算的 Q 矩阵转换为 Kaiwu 库中的 QUBO 格式
153. import numpy as np
154. import kaiwu as kw
155.
156. # 将之前计算的 Q 矩阵转换为 Kaiwu 库中的 QUBO 格式
157. matrix = np.array([[2097, 0, 0, 0],
158.                     [0, 3140, 0, 0],
159.                     [0, 0, 5200, 0],
160.                     [0, 0, 0, 6320]])
161.

```

```

162. # 将 QUBO 矩阵转换为 Ising 矩阵
163. ising_mat, _ = kw.qubo.qubo_matrix_to_ising_matrix(matrix)
164.
165. ising_matrix = kw.cim.adjust_ising_matrix_precision(_matrix)
166. kw.qubo.qubo_matrix_to_qubo_model(matrix)
167.
168. QUBO 矩阵转换为 Ising 矩阵
169. import numpy as np
170. import kaiwu as kw
171. matrix = np.array([[2097, 0, 0, 0],
172.                    [0, 3140, 0, 0],
173.                    [0, 0, 5200, 0],
174.                    [0, 0, 0, 6320]])
175. # 将 QUBO 矩阵转换为 Ising 矩阵
176. matrix, _ = kw.qubo.qubo_matrix_to_ising_matrix(matrix)
177. # 调整精度
178. ising_matrix = kw.cim.adjust_ising_matrix_precision(matrix)
179. rng = np.random.default_rng(10)
180. solver = kw.cim.SimulatedCIMOoptimizer(pump=1.0,
181.                                         noise=0.1,
182.                                         laps=1000,
183.                                         delta_time=0.1,
184.                                         normalization=0.5,
185.                                         iterations=10,
186.                                         size_limit=10,
187.                                         rng=rng)
188.
189. solver.solve(ising_matrix)
190. 模拟退火求解
191. import numpy as np
192. import matplotlib.pyplot as plt
193.
194. def objective(w):
195.     return -(2000*w[0] + 3000*w[1] + 5000*w[2] + 6000*w[3])
196.
197. def constraint(w):
198.     return 100*w[0] + 140*w[1] + 200*w[2] + 320*w[3] <= 2400 and
199.     np.sum(w >= 1) >= 3
200. def simulated_annealing(objective, bounds, n_iterations, step_size,
201. temp):
202.     # 随机初始解
203.     best = np.random.randint(low=0, high=10, size=len(bounds))
204.     best_eval = objective(best)
205.     curr, curr_eval = best, best_eval
206.     scores = [best_eval]
207.     for i in range(n_iterations):
208.         # 在邻域中取一个点
209.         candidate = curr + np.random.randn(len(bounds)) * step_size

```

```

209.     candidate = np.clip(candidate, a_min=[b[0] for b in bounds],
a_max=[b[1] for b in bounds])
210.     candidate = np.round(candidate).astype(int) # 确保是整数
211.     # 检查约束
212.     if not constraint(candidate):
213.         continue
214.     # 计算新点的目标函数值
215.     candidate_eval = objective(candidate)
216.     # 检查是否接受新点
217.     if candidate_eval < best_eval:
218.         best, best_eval = candidate, candidate_eval
219.         scores.append(best_eval)
220.         curr, curr_eval = candidate, candidate_eval
221.     else:
222.         diff = candidate_eval - curr_eval
223.         t = temp / float(i + 1)
224.         metropolis = np.exp(-diff / t)
225.         if np.random.rand() < metropolis:
226.             curr, curr_eval = candidate, candidate_eval
227.     # 降温
228.     temp *= 0.99
229.     return best, best_eval, scores
230.
231. # 参数设置
232. bounds = [(0, 100)] * 4 # w1, w2, w3, w4 的取值范围
233. n_iterations = 1000
234. step_size = 1.0
235. temp = 10.0
236.
237. # 运行模拟退火算法
238. best, score, scores = simulated_annealing(objective, bounds,
n_iterations, step_size, temp)
239.
240. # 输出结果
241. print('Best Solution: %s' % best)
242. print('Best Objective: %s' % -score)
243.
244. # 可视化
245. plt.plot(scores, '-.')
246. plt.xlabel('Improvement Number')
247. plt.ylabel('Objective Function Value')
248. plt.show()
249.
1. 迭代法求解问题二
250. import numpy as np
251.
252. # 初始化矩阵
253. t = np.zeros((4, 24))

```

```

254. for i in range(4):
255.     t[i] = np.arange(1, 25)
256.
257. z = np.array([190, 175, 165, 150]).reshape(4, 1)
258. z = np.broadcast_to(z, (4, 24))
259.
260. d = np.array([0.9, 1.2, 1.8, 2.1]).reshape(4, 1)
261. d = np.broadcast_to(d, (4, 24))
262.
263. s = np.array([100, 140, 200, 320]).reshape(4, 1)
264. s = np.broadcast_to(s, (4, 24))
265.
266. y = np.array([28, 30, 34, 38]).reshape(4, 1)
267. y = np.broadcast_to(y, (4, 24))
268.
269. r = np.array([7000, 7500, 8500, 9000]).reshape(4, 1)
270. r = np.broadcast_to(r, (4, 24))
271.
272. rp = np.array([1000, 1500, 2000, 3000]).reshape(4, 1)
273. rp = np.broadcast_to(rp, (4, 24))
274.
275. # 定义目标函数
276. def objective_function(w, t, z, d, s, y, r, rp):
277.     term1 = w**2 * t * ((192000 * z * d) - s + 9600 * y + 60 * r +
278.     60 * rp)
279.     return np.sum(term1) - 13185600
280. # 检查约束
281. def check_constraints(w, t, k_vals):
282.     constraints = [
283.         np.dot(w[0], t[0]) == k_vals[0],
284.         np.dot(w[1], t[1]) == k_vals[1],
285.         np.dot(w[2], t[2]) == k_vals[2],
286.         np.dot(w[3], t[3]) == k_vals[3]
287.     ]
288.     return all(constraints)
289.
290. # 遍历所有 k 值的组合
291. k_range = range(10)
292. max_value = float('-inf')
293. best_w = None
294. best_ks = None
295.
296. for k11 in k_range:
297.     for k12 in k_range:
298.         for k13 in k_range:
299.             if k11 + k12 + k13 == 7:
300.                 for k21 in k_range:
301.                     for k22 in k_range:

```

```

302.             for k23 in k_range:
303.                 if k21 + k22 + k23 == 7:
304.                     for k31 in k_range:
305.                         for k32 in k_range:
306.                             for k33 in k_range:
307. if k31 + k32 + k33 == 3:
308. k_vals = [
309. k11,
310. 0.5 * k12 + k21,
311. 0.5 * k13 + 0.5 * k22 + k31,
312. 0.5 * k23 + k32
313.         ]
314. w = np.zeros((4, 24))
315. for row in range(4):
316. best_row_value = float('-inf')
317. best_col = 0
318. for col in range(24):
319. w_temp = w.copy()
320. w_temp[row, :] = 0
321. w_temp[row, col] = 1
322. if check_constraints(w_temp, t, k_vals):
323.     current_value = objective_function(w_temp, t, z, d, s, y, r, rp)
324.     if current_value > best_row_value:
325.         best_row_value = current_value
326.         best_col = col
327.         w[row, :] = 0
328.         w[row, best_col] = 1
329.         current_value = objective_function(w, t, z, d, s, y, r, rp)
330.         if current_value > max_value:
331.             max_value = current_value
332.             best_w = w.copy()
333.             best_ks = [k11, k12, k13, k21, k22, k23, k31, k32, k33]
334. print("最佳 k 值组合:", best_ks)
335. print("优化后的 w 矩阵:")
336. print(best_w)
337. print("目标函数最大值为:", max_value)
338. CMI 和 模拟退火求解
339. import pulp as pl
340. # 创建问题实例
341. model = pl.LpProblem("Maximize_Profit", pl.LpMaximize)
342. # 定义变量：普卡数量
343. x = {i: pl.LpVariable(f'x_{i}', lowBound=0, cat='Integer') for i in
344. range(1, 5)}
345. # 普卡参数
346. params_excavator = {
347.     1: {'efficiency': 190, 'price': 100, 'labor': 7000,

```

```

'maintenance': 1000, 'fuel': 28},
348. 2: {'efficiency': 175, 'price': 140, 'labor': 7500,
'maintenance': 1500, 'fuel': 30},
349. 3: {'efficiency': 165, 'price': 200, 'labor': 8500,
'maintenance': 2000, 'fuel': 34},
350. 4: {'efficiency': 150, 'price': 320, 'labor': 9000,
'maintenance': 3000, 'fuel': 38}
351. }
352.
353. # 卡车数量
354. trucks = {1: 7, 2: 7, 3: 3}
355.
356. # 卡车匹配变量
357. truck_match = {
358. 1: {1: pl.LpVariable('match_1_1', lowBound=0, cat='Integer')},
359. 2: {1: pl.LpVariable('match_2_1', lowBound=0, cat='Integer'), 2:
pl.LpVariable('match_2_2', lowBound=0, cat='Integer')},
360. 3: {1: pl.LpVariable('match_3_1', lowBound=0, cat='Integer'), 2:
pl.LpVariable('match_3_2', lowBound=0, cat='Integer'), 3:
pl.LpVariable('match_3_3', lowBound=0, cat='Integer')},
361. 4: {2: pl.LpVariable('match_4_2', lowBound=0, cat='Integer'), 3:
pl.LpVariable('match_4_3', lowBound=0, cat='Integer')}
362. }
363.
364. # 目标函数和成本计算
365. revenue_per_m3 = 20
366. working_days = 160
367. years = 5
368. fuel_price = 7
369.
370. # 计算总收入
371. total_revenue = sum(
372. x[i] * params_excavator[i]['efficiency'] * revenue_per_m3 *
working_days * 12 * years for i in range(1, 5)
373. )
374.
375. # 计算总成本
376. total_cost = sum(
377. x[i] * (params_excavator[i]['price'] +
(params_excavator[i]['labor'] + params_excavator[i]['maintenance']) * 12 *
years + params_excavator[i]['fuel'] * working_days * 12 * years *
fuel_price) for i in range(1, 5)
378. )
379.
380. model += total_revenue - total_cost, "Profit"
381.
382. # 约束条件
383. model += sum(x[i] * params_excavator[i]['price'] for i in range(1,
5)) <= 2400 # 初始资本约束

```

```

384. model += sum(x[i] for i in range(1, 5)) >= 3
385.
386. # 添加约束，确保未租赁的普卡不能配备
387. for i in truck_match:
388.     for j in truck_match[i]:
389.         model += truck_match[i][j] <= x[i] * trucks[j],
f"Link_Excavator_{i}_Truck_{j}"
390.
391. # 特定的匹配约束实现
392. model += truck_match[1][1] >= x[1],
"Match_Excavator_1_with_Truck_1" # 卡 1 至少一辆货 1
393. model += 2 * truck_match[2][1] + truck_match[2][2] >= x[2],
"Match_Excavator_2_with_Trucks_1_or_2" # 卡 2 至少两辆卡 1 或一辆卡 2
394. model += 2 * truck_match[3][1] + 2 * truck_match[3][2] +
truck_match[3][3] >= x[3], "Match_Excavator_3_with_Trucks_1_2_or_3" # 卡 3
至少需要相应的卡车
395. model += 2 * truck_match[4][2] + truck_match[4][3] >= x[4],
"Match_Excavator_4_with_Trucks_2_or_3" #4 至少需要相应的卡车
396.
397. # 解决问题
398. model.solve()
399.
400. # 打印结果
401. if model.status == pl.LpStatusOptimal:
402.     print("最优解:")
403.     print(f"总收益: {pl.value(model.objective)} 万元")
404.     for i in range(1, 5):
405.         print(f"普卡 {i} 的数量: {x[i].varValue}")
406.         if i in truck_match:
407.             for j in truck_match[i]:
408.                 print(f"  配备卡车 {j} 的数量:
{truck_match[i][j].varValue}")
409.     else:
410.         print("没有找到最优解")
411.

```